

Security Audit Report: Binance MCP Server

Executive Summary

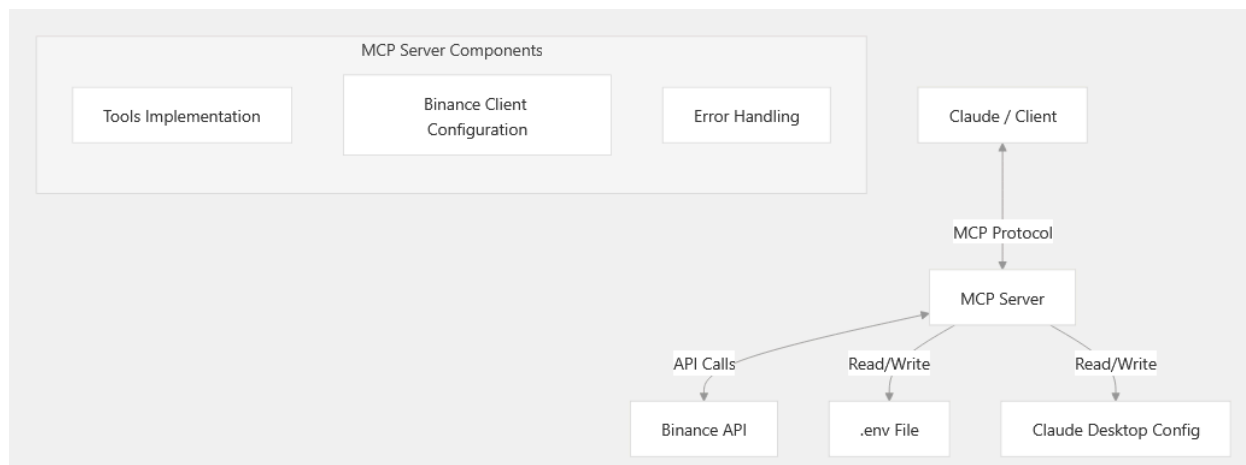
This security audit evaluates the Binance MCP Server repository, a Node.js application designed to interact with the Binance API for cryptocurrency trading and account management. The application uses the Model Context Protocol (MCP) framework to facilitate secure, structured transactions with the Binance exchange. The audit identified several security concerns related to credential management, error handling, input validation, and other security best practices.

Risk Summary

Risk Level	Count
Critical	2
High	3
Medium	4
Low	3

System Architecture

The Binance MCP Server is structured as follows:



Findings

1. Critical: Insecure Storage of API Credentials

Description: The application stores Binance API keys and secrets in plaintext in both the .env file and Claude Desktop configuration file. These credentials provide full access to a user's Binance account and funds.

Evidence:

The initialization script saves credentials to an unencrypted .env file: `init.ts:61-68`

The same credentials are also stored in the Claude Desktop configuration file: `init.ts:72-86`

Risk Level: Critical

Recommendation:

- Implement secure credential storage using environment-specific secret management solutions
- Consider using a secure password-protected keychain or vault
- Never write API credentials to unencrypted configuration files
- Consider implementing a secure token exchange mechanism rather than storing raw credentials

2. Critical: Inadequate Access Controls

Description: The application lacks any authentication mechanism for the MCP server itself. Any client that can connect to the MCP server can access all functionality without additional verification.

Evidence:

The server setup and connection have no authentication mechanisms:
index.ts:23-48

Risk Level: Critical

Recommendation:

- Implement authentication for the MCP server
- Add session management and token-based authentication
- Implement request signing for secure communication
- Consider adding IP restrictions for connections

3. High: Excessive Error Information Disclosure

Description: The application returns detailed error messages directly from the Binance API, potentially exposing sensitive information about the API structure, account details, or implementation specifics.

Evidence:

Error messages are returned without sanitization: binanceAccountInfo.ts:46-54

Similar pattern in order placement: binanceSpotPlaceOrder.ts:37-45

Risk Level: High

Recommendation:

- Implement error sanitization to avoid leaking internal details
- Create standardized error responses with appropriate logging
- Log detailed errors server-side only
- Return generic error messages to clients

4. High: Insufficient Input Validation

Description: While the code uses Zod for schema validation, many parameters lack thorough validation for security-critical operations like order placement.

Evidence:

Order placement has basic type validation but lacks comprehensive validation:
newOrder.ts:10-22

Risk Level: High

Recommendation:

- Enhance input validation with more comprehensive rules
- Implement validation for numerical ranges, string patterns, and enumerations
- Add business logic validation beyond basic type checking
- Consider adding a validation middleware layer

5. High: No Rate Limiting

Description: The application does not implement rate limiting, potentially allowing clients to flood the server with requests or perform high-frequency trading operations that could lead to account restrictions or financial losses.

Evidence:

No rate limiting mechanisms were found in the codebase.

Risk Level: High

Recommendation:

- Implement request rate limiting
- Add cooldown periods for critical operations
- Consider implementing progressive backoff for repeated requests
- Add monitoring for suspicious activity patterns

6. Medium: Lack of Secure Configuration Options

Description: The configuration process doesn't offer secure options for handling credentials, and the setup doesn't include security best practices guidance.

Evidence:

The initialization script focuses on functionality without security considerations:
init.ts:134-159

Risk Level: Medium

Recommendation:

- Add security-focused configuration options
- Implement a secure setup wizard
- Provide documentation on secure deployment
- Include security warnings during setup

7. Medium: No Logging Strategy

Description: The application lacks a comprehensive logging strategy, making it difficult to audit actions, debug issues, or detect security incidents.

Evidence:

Only basic error output is present, with no structured logging system:

getAccount.ts:31-38

Risk Level: Medium

Recommendation:

- Implement a structured logging system
- Add audit logs for security-critical operations
- Ensure logs don't contain sensitive information
- Add log rotation and retention policies

8. Medium: Unprotected Sensitive API Operations

Description: Sensitive operations like placing orders or accessing account information don't have additional confirmation or verification steps.

Evidence:

Order placement lacks additional verification: newOrder.ts:23-52

Risk Level: Medium

Recommendation:

- Add confirmation requirements for sensitive operations
- Implement transaction limits

- Consider two-factor authentication for critical actions
- Add notifications for high-value transactions

9. Medium: Dependency Management Concerns

Description: The application relies on numerous Binance SDK packages without version pinning or security scanning.

Evidence:

Multiple Binance dependencies in package.json: package.json:59-76

Risk Level: Medium

Recommendation:

- Pin dependency versions explicitly
- Implement dependency security scanning
- Set up automated vulnerability checks
- Create a dependency update policy

10. Low: No Sanitization of API Responses

Description: API responses are returned directly to clients without sanitization, potentially exposing unnecessary information.

Evidence:

Full response data is returned: getAccount.ts:23-28

Risk Level: Low

Recommendation:

- Sanitize API responses to include only necessary data
- Create response transformation layers
- Implement data minimization practices
- Consider implementing a response schema validation

11. Low: No Transport Layer Security Configuration

Description: The code doesn't specify or enforce transport layer security configurations.

Evidence:

Server connection doesn't specify security options: index.ts:46-47

Risk Level: Low

Recommendation:

- Enforce encrypted communications
- Specify minimum TLS versions
- Configure secure cipher suites
- Implement certificate validation

12. Low: No Security Headers or Protection Mechanisms

Description: The application doesn't implement common security headers or protection mechanisms.

Evidence:

No security headers are set in the transport layer configuration.

Risk Level: Low

Recommendation:

- Implement appropriate security headers
- Add protection against common web vulnerabilities
- Consider adding request signing for all operations
- Implement secure session management

Conclusion

The Binance MCP Server provides a functional interface to the Binance API but has several security weaknesses that should be addressed before deployment in a production environment. The most critical issues involve credential management and access controls, which could lead to unauthorized access to trading accounts

and potential financial losses.

Recommended Action Plan

Immediate Actions:

- Secure storage of API credentials
- Implement basic authentication for the MCP server
- Improve error handling and sanitization

Short-term Actions:

- Enhance input validation
- Implement rate limiting
- Add logging infrastructure

Medium-term Actions:

- Improve configuration security
- Implement response sanitization
- Enhance transport security

Long-term Security Improvement:

- Comprehensive security testing
- Implement a security monitoring strategy
- Regular security audits and updates

Notes

This audit focused on the code-level security issues visible in the repository. A complete security assessment would also include runtime testing, dependency scanning, and infrastructure evaluation. The repository appears to be a work in progress with several features marked as incomplete, which presents an opportunity to implement security improvements during development.